

**Machine learning, neural networks and econometric
models for prediction the realized volatility of Bitcoin and
E-mini SP500**

Manevich Vyacheslav^a (vmanevich@hse.ru), Ignatov Dmitriy^a
(dmitrii.ignatov@gmail.com)

^a Russia, Moscow, National Research University Higher School of Economics; Postal
address: 11 Pokrovsky Boulevard, Moscow, Russia, 109028.

Corresponding Author:

Manevich Vyacheslav

Russia, Moscow, National Research University Higher School of Economics (HSE
University); Postal address: 11 Pokrovsky Boulevard, Moscow, Russia, 109028.

Tel: +7 (996) 376-7167

Email: vmanevich@hse.ru

Machine learning, neural networks and econometric models for prediction the realized volatility of Bitcoin and E-mini SP500

Manevich Vyacheslav^{a,1}, Ignatov Dmitriy^a

^a*Russia, Moscow, National Research University Higher School of Economics (HSE University); Postal address: 11 Pokrovsky Boulevard, Moscow, Russia, 109028.*

Abstract

The article compares ARMA, boosting, neural network models, GARCH and HAR models, and proposes a new efficient method for forecasting one day ahead of the realized volatility of financial series. The GARCH and HAR models are taken as comparable models, which showed the best results in Manevich et al. (2023). In addition, the transfer learning phenomenon for boosting and neural network models is investigated. Bitcoin and E-mini S&P 500 are chosen as examples. Realized volatility is calculated from intraday (intraday - 24 hours) data. The calculation is based on the closing values of the internal five-minute intervals. All days of the week are almost equally present among the days selected for forecasting. The comparison is made on the observation interval from 01.01.2018 to 29.12.2021. Since there could be (and have been) structural changes in the markets during this interval, the models are evaluated in sliding windows of 399 days length. For each time series, we compare three-parameter enumeration boostings, about 10 different neural network architectures, ARMA models, a newly designed method, and different transfer learning and training sample expansion options. It is shown that GARCH and HAR models are inferior to other listed methods and models. CNN architecture is the most suitable for financial time series forecasting. The proposed method shows the

*Corresponding author.

Email address: vmanevich@hse.ru (Manevich Vyacheslav)

results comparable to neural networks and boostings in precision. Transfer learning shows improvement in some cases, especially for neural networks, and needs to be studied in more detail. ARMA models outperform other listed methods in some cases. The smallest relative errors for the Bitcoin and E-mini S&P 500 realized volatility predictions are 25.28% and 30.74%, respectively.

Keywords: Time Series, Forecasting, Bitcoin, E-mini S&P 500, Cryptocurrency, Realized Volatility, Transfer Learning, Convolution Neural Network, Boosting.

1. Introduction

Forecasting the values of an exchange-traded asset is a complex task. It is also an important task to forecast the volatility of an asset, identifying its fluctuations in the future. Volatility forecasting is important in terms of making a trading strategy, as well as for decision-making organizations and institutions such as the Central Bank. This article has investigated the possibilities of predicting the realized volatilities of Bitcoin and E-mini S&P 500 using neural network models, machine learning methods and a new efficient method. Furthermore, there was a comparison of obtained errors with the errors of classical HAR-RV and GARCH models presented in Manevich et al. (2023).

Manevich et al. (2022) confirmed the presence of volatility spillovers between the two mentioned assets, which are among the main representatives of the cryptocurrency and classical stock exchanges. Neural network models, among others, allow us to exploit this connectivity in training. For example, by using two time series simultaneously training or by applying variations of transfer learning.

As shown by researchers (Dutta et al. (2021); Wang et al. (2021); Singhal and Swarup (2011)) high volatility of futures prices creates big obstacles for volatility forecasting, in the case of using oil and electricity futures volatilities.

Kristjanpoller and Minutolo (2016) proposed a hybrid ANN-GARCH model to predict Euro/Dollar and Yen/Dollar volatility. The authors considered a

small number of layers and neurons for the ANN part and concludes about the best type of architecture. The hybrid approach was shown to be efficient and reduce errors.

Two years later, Kristjanpoller and Minutolo (2018) proposed the application of the developed ANN-GARCH model to Bitcoin volatility. Unfortunately, the authors didn't provide the results obtained for the hybrid MAPE model, but the relative errors for different types of GARCH models are large, which is also consistent with our results.

Fisher and Krauss (2017) investigated the applicability of LSTM models in forecasting using the S&P 500 returns from 1992 to 2015 as an example. The authors showed the effectiveness of LSTM relative to RNN and random forest (in some cases).

Kim and Won (2018) used hybrid models to predict stock price volatility. They combined a machine learning model, namely the Long-Term Short-Term Memory (LSTM) model with some of the GARCH models. The proposed methodology improved the prediction performance compared to the GARCH models.

Shen Z. et al. (2021) compared RNN (recurrent neural network) models, GARCH and EWMA on Bitcoin data. The authors show that RNNs are better in average prediction efficiency, but they don't capture Bitcoin outliers well enough.

Zolfaghari et al. (2021) used the Dow Jones Industrial Average (DJIA) and Nasdaq Composite (IXIC) to investigate a large number of hybrid models based on ARMAX, GARCH and LSTM models, and the applicability of wavelet transformations. The results show an overall improvement in stock index prediction using the AWT-LSTM-ARMAX-FIEGARCH model with the Student's t distribution. A robust test proves that this model has better prediction accuracy at different time horizons (1-, 10-, 15-, 20-, 30-, and 60-day ahead) for both stock indices. Also, AWT-LSTM improves the ability of the HAR (3) X-RV model in predicting realized stock volatility for the specified time horizons.

Shusheng et al. (2022) used data from the Chinese oil futures market and

the Australian electricity futures market to find that the accuracy of XGBoost volatility forecasting is significantly superior to GARCH-jump and HAR-jump models in both markets.

Livieris et al. (2020) used LSTM and CNN-LSTM models to predict gold price volatility. The study showed improved prediction results when using a combination of CNN and LSTM layers.

Livieris et al. (2021) applied a model based on LSTM layers and ensembles to account for the features of the cryptocurrencies under study, Bitcoin, Ethereum and Ripple. The proposed model can leverage mixed cryptocurrency data, reduce overshoot and provide lower computing cost compared to a traditional deep neural network in terms of fewer weights (and lower computation time).

Wang and Guo (2020) combined different methods and proposed a hybrid DWT-ARIMA-GSXGB model to achieve improved prediction precision, approximation and generalization abilities. First, a discrete wavelet transform is applied to split the data set into approximation and error parts. Then the ARIMA (0, 1, 1), ARIMA (1, 1, 0), ARIMA (2, 1, 1) and ARIMA (3, 1, 0) models process approximate partial data, and the advanced xgboost (GSXGB) model process erroneous partial data. Finally, the prediction results are combined using wavelet reconstruction.

The aim of this article is to compare classical, neural network and machine learning methods for the financial time series prediction problem. Moreover, the purpose is to study the transfer learning in predicting the time series of the realized volatilities of Bitcoin and S&P 500. This article compares a large number of models – both classical (ARIMA, HAR, regression) and machine learning (boosting), various neural network architectures, approaches to learning – transfer learning, feature generation for sample expansion and developed a new CTCM model, which is described in section 3.9. Thus, this article investigates a variety of models and methods, proposes a new efficient forecasting method and shows the effectiveness of transfer learning for financial time series. In addition, this study not only provides other researchers with a good base of precedents,

but also with Python code (posted on the githab¹) that can easily be reused.

The article has a structure: the section “Data” provides a detailed overview of the data used, as well as formulas for calculating the realized volatility. The next “Methodology” chapter contains a description of the models, methods and approaches used in the article, as well as methods and models that were not included in the article due to insufficient forecast accuracy. Section “Results” contains tables with errors of the best chosen models, as well as forecast charts and error analysis and “Conclusion” is the final section, where the results and conclusions are given.

2. Data

2.1. Data

The study considers the period from 01.01.2018 to 29.12.2021 (inclusive), all data is taken from finam.ru², the values at the closing of internal five-minute intervals are used. The interval of 399 days in the rolling window of length 5 is considered for forecasting. In other words, every fifth value is forecasted. This approach allows reducing the program’s running time and considering almost the same proportions of forecasts for different days of the week.

Exchange instruments are used for analysis:

- Bitcoin (btc) is the cryptocurrency with the largest capitalization, we can say that it is the “main” cryptocurrency at the moment. It is traded 24/7.
- The E-mini S&P500 Index (snp) is a futures contract traded on the Chicago Mercantile Exchange (CME) representing one fifth of the value of a standard S&P 500 index futures contract. The S&P500 Index includes: 400 industrial corporations, 20 transportation corporations, 40 financial corporations and 40 utilities. Almost all the companies are traded on the NYSE, but there are also those traded on the AMEX and Nasdaq. The

¹https://github.com/VyacheslavMan/Article_2

²<https://www.finam.ru/profile/cryptocurrencies/btc-usd/export>

base asset for this futures is the value of the S&P 500 stock index. This futures is valuable to research as it is related to one of the largest S&P 500 indices and trades almost all day. It is traded from 6:00 Sunday to 5:00 Friday (Chicago Stock Exchange time) with a daily break from 5:00 to 6:00.

Fig.1 shows a graph of sample average values of returns calculated in a sliding window of length 399 with a sliding step of 5. The btc yields are characterized by quite strong changes, a sharp rise and fall, although in general there is an uptrend. At the same time, there are no such sharp and frequent jumps for futures yields in general, except for the periods from March to April 2020 and May to June 2021.

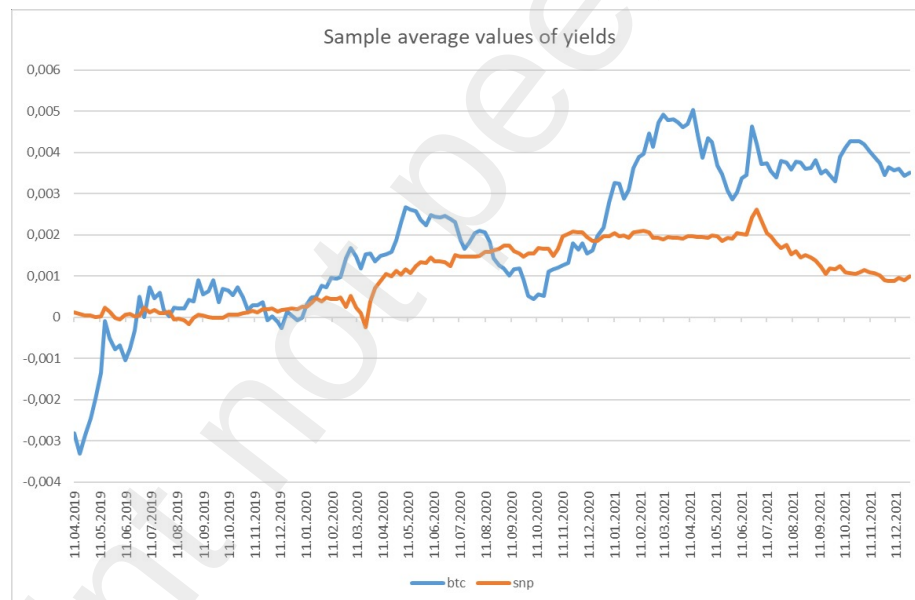


Figure 1: The blue line – sample averages for Bitcoin yields, the red line – sample averages for futures yields.

Fig. 2 shows at the sample variance of returns calculated in a sliding window of length 399 with a sliding step of 5. Here we observe a significantly higher

variance for btc and a rather small variance for futures. For btc, there is a sharp jump from March to April 2020 and a fall from May to June 2021. At the same time, for futures, there is a slight rise from May to June 2021. This may indicate some similarities in the processes and perhaps they will become more similar in the future.

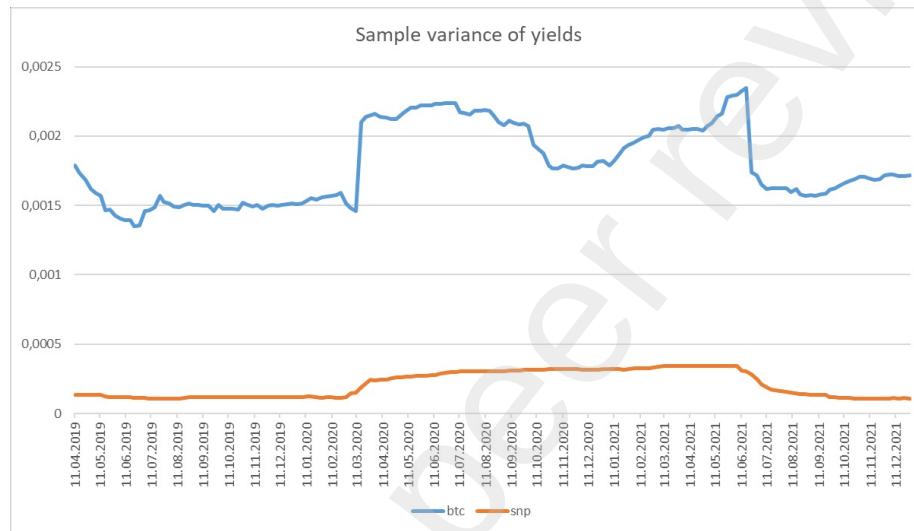


Figure 2: The blue line – the sample variance for Bitcoin yield, the red line – the sample variance for the futures yield.

Table 1 shows the frequency of encountering days of the week among the days for which the forecast is made. The most frequently encountered day is Wednesday, that is, the middle of the week. However, in general, all the days occur an almost equal number of times, the maximum difference is 4. These statistics show that the forecast falls on all trading days of the week almost equally, which makes the results obtained in this article representative.

Day of the week	Frequency of encounter	Percentage of sample
Monday	27/170	15.9%
Tuesday	27/170	15.9%
Wednesday	31/170	18.2%
Thursday	30/170	17.6%
Friday	28/170	16.5%
Sunday	27/170	15.9%

Table 1: Frequency of days of the week.

2.2. Realized volatility

The five-minute realized volatility is used as the realized volatility. The choice of such time intervals is due to the study of Bollerslev (1986). They showed that the values of the realized volatility calculated by five-minute intervals are the most optimal in terms of accuracy and microstructure errors. Previously, the same form of volatility calculation was used in the articles of Manevich et al. (2022) and Manevich et al. (2023).

The realized volatility on day t is presented as:

$$RV_{t,j} = \sqrt{\sum_{j=1}^N r_{t,j}^2}, \quad (1)$$

where $r_{t,j} = \log(p_{t,j}) - \log(p_{t,j-1})$ – the yield, $p_{t,j}$ – the price of the asset on day t at the end of the intraday interval j of length T (seconds), $j = 1 \dots N$ with the total number of intervals for one day equal to N . The first five minutes are 00:00 – 00:05. The last five minutes are 23:55 – 00:00. Calculation is done by closing prices of five-minute periods.

The RV described above is the square root of the realized variance. In HAR models, realized variance is predicted and then the square roots are taken from them to obtain realized volatilities.

In the article, to obtain comparable results in the presence of gaps in the data, the realized volatility is calculated as follows:

- If there are less than 5 hours of data in the day, the day is deleted;

- If observations are missing at the beginning and/or at the end of the day, the realized volatility is calculated using the available K five-minute intervals, and then scaled to the daily data. Since the day contains 288 five-minute intervals, the numerator is 288:

$$RV_t = \sqrt{\frac{288}{K} \sum_{j=1}^K r_{t,j}^2}. \quad (1.1)$$

- In the case of missing data within a day, for example, between moments j_1 and j_2 , the appropriate sum is replaced by the square of the yield for the missed period.

Thus, the comparability of the daily realized volatility values of the futures on different days with the daily realized volatility values of Bitcoin is obtained.

In Manevich et al. (2022) it was obtained that the realized volatilities are of lognormal nature and it is the HAR-In(RV) models that show the best prediction precision. In this article, it is the logarithms of the realized volatilities that are fed into the input of the models, and then the exponents from the forecasts are taken. Thus, we can take into account the lognormal character of the realized volatilities and obtain the highest-quality forecasts of the studied values.

3. Methodology

3.1. Workflow

A large number of models and approaches were investigated and compared in the article, the tables in the “Results” section present the best of them by MAPE indices (see (2)). Also in section 3.5 we present models which did not show good prediction accuracy, but which were also built and investigated. The article provides, according to the authors, an exhaustive comparison (in terms of prediction accuracy) of neural network models and boosting (with/without Transfer Learning) with similar predictions of classical GARCH and HAR models from Manevich et al. (2023), as well as with the CTCM (item 3.9) and $ARIMA(p, 0, q)$ models, where $p \in [1, 10]$, $q \in [0, 9]$. The Accuracy scores (see

(3)) for all models are also compared. All models are trained in a sliding window of 399 days with a sliding step of 5, followed by a 1-step-ahead forecast. Thus, all models make predictions for the same days, using the same data.

For repeatability, `numpy.random.seed(7)` is used because the neural networks give slightly different results with each training. Each model in each window is trained 10 times (each time a prediction is made), the prediction for each step is averaged, and then MAPE is calculated, which allows to achieve relative repeatability of the results.

Since the realized volatility is predicted, and Manevich et al. (2023) confirmed the logarithmic nature of the realized volatility, the logarithms of the realized volatility are input into the models, and exponents are taken from the resulting predictions. Accuracy metrics are also compared, that is, how often the models guessed the direction of the process change.

After the MAPEs for the neural networks are calculated, the errors are compared with the MAPEs of the selected best HAR- $\ln(RV)$ models, conclusions are made about the best models and the efficiency/non-efficiency of the classical models.

After selecting the models with the best MAPE values, the optimal feed window length is investigated on the selected models. The enumeration is performed using the Optuna package, which uses Bayesian optimization techniques to reduce running time, as well as classical grid-based enumeration. Optuna does not always give reproducible results because it randomly selects starting points for the optimization process. However, all selected models are almost equally efficient. For neural networks, since in their case it is rather difficult to obtain reproducible results, the check of the optimal window length is done by several values of the window lengths that are selected as optimal in other models.

3.2. Boosting

This article describes forecasting with the help of the “xgboost” package and the function “XGBRegressor” in Python. A search is carried out according to

3 parameters, $n_estimators = 100$, 5 elements in each sample:

1. $\eta = [0.1, 1]$ in steps of 0.1;
2. $\gamma = [0.1, 1]$ in steps of 0.1;
3. $max_depth = [1, 11]$ in steps of 1.

In the following tables, the results of the boosts are named “xgboost_(eta)_(gamma)_(max_depth)”, where instead of parameters are their values in the model.

Catboost and adaboost are also considered. Their results are almost identical (and sometimes inferior) to those of xgboost and are not given in the tables below.

3.3. Architectures of neural networks

As a result of the search, it was found that the best performance of the models is achieved with the “*tanh*” activation function – the tables show the results of the models with this activation function.

For all presented and mentioned below neural network models, the loss function of training is the classical MAE. Empirically on the given data sets, it is obtained that with loss=“mae” the best error rates are obtained. The batch_size parameter, as well as the batch lengths (lags), are chosen to be 5. For training in each sliding window 200 epochs are taken with control of training by unchanging loss on 10 consecutive epochs, optimizer= “adam”.

The article focuses on neural networks with one gate and one or two feature-rows (features) on the gate (in the endnotes “TS.1” and “TS.2”, respectively). Many architectures such as LSTM (Hochreiter S. and Schmidhuber J. (1997)), BiLSTM, CNN-LSTM, MLP, RNN, CNN and Encoder-Decoder are investigated. The most efficient of them, in terms of MAPE and running time, were the CNNs. At the same time, the number of layers has little effect on the error, which can be attributed to the features of the data and training, not only to the predictive abilities of the models themselves. It is also worth noting that the models with the best MAPE did not always have the best Accuracy, but the

difference in accuracy in guessing the direction of change is small. The architectures of the two models that showed the best results in MAPE and Accuracy for both data series are described below. They are presented in the following tables. The LSTM-based encoder decoder is architecturally similar to CNN, and its results are slightly lower than CNN in precision and running time.

- First CNN with 1 layer (M_1_CNN):
filters=4features, kernel_size=lags;
dense_layer_output: features neurons.
- Second CNN with 1 layer (M_2_CNN):
filters=features, kernel_size=lags;
dense_layer_output: features neurons.

3.4. *Transfer Learning*

This section deals with Transfer Learning (TL) only for the above two architectures: M_1_CNN and M_2_CNN, as well as for boosting. The following TL variants are investigated, with appropriate labels at the end of the model names:

1. Training on RV Bitcoin and RV E-mini S&P500 forecasting – “tbs”;
2. RV E-mini S&P500 Training and RV Bitcoin Forecasting – “tsb”;
3. Training on RV Bitcoin and RV E-mini S&P500 followed by forecasting both assets – “tbbss” (for forecasting btc) or “tssbb” (for forecasting snp).

This approach presumably can help to slightly improve the results relative to the results obtained in 5.1. and 5.2. because, firstly, assets, in general, have similar behavior patterns, but one of them is more volatile, and secondly, it allows expanding the size of the training sample, which can have a positive effect on prediction precision. Also, it can be assumed that assets have different propensity and ability to transfer learning and it makes sense to transfer learning from a network trained on more volatile assets to improve prediction results for less volatile assets. A similar assumption can be made about methods. It is

likely that neural networks are better adapted to transfer learning than boosting models due to their inherent architectural features.

3.5. Other methods

A large number of models, architectures, and approaches have been studied in the course of writing this article. Not all of them have shown effectiveness and good results. The following models and approaches were investigated in the article, but not included among the best, so their results are not presented in tables and graphs:

1. Classic regression, similar to the AR(1) process. Errors are 4-5% higher than in CNN. Running time is many times less than CNN.
2. RNN. MAPE is 4-5% higher than CNN. The running time is 1.5 times more than that of CNN.
3. LSTM. MAPE is 9-10% higher than CNN. The running time is 2-3 times longer run time than the CNN.
4. BiLSTM. MAPE is 9-10% higher than CNN. The running time is 2 times longer run time than the CNN.
5. LSTM – CNN. MAPE is 4-5% higher than CNN. The running time is 2-3 times longer run time than the CNN.
6. BiLSTM – CNN. MAPE is 4-5% higher than CNN. The running time is 2 times longer run time than the CNN.
7. Encoder – decoder based LSTM. The model is architecturally similar to the classical CNN, it is % less precision than CNN. The running time is 1.5-2 times longer run time than the CNN.
8. MLP (2 to 15 layers with 10/16/25/32/64 neurons) – MAPE is 5-6% higher than CNN. The running time is 1.5 times longer run time than the CNN.
9. Library TabNet. Prediction results made with this library with enumeration of different internal parameters are 5-6% inferior to selected CNN models. Running time is comparable to CNN.

10. The methods listed above and below with artificial expansion of the training sample using the tsfresh library (Christ et al. (2018)) are inferior to the corresponding models without artificial expansion of the training sample set by 4-5%. This library is interesting because it allows the extraction of features relevant to a given time series and its target. More than 4,000 features were extracted in each sliding window step, and then only 22 relevant ones were selected. The features extracted in each sliding window were fed into the models as well as into “tbbss”/“tssbb” type transfer learning methods. However, this did not help to improve model results and increased running time.
11. The methods listed above and below with artificial expansion of the training sample using the tsfresh library (Christ et al. (2018)) are inferior to the corresponding models without artificial expansion of the training sample set by 4-5%. This library is interesting because it allows the extraction of features relevant to a given time series and its target. More than 4,000 features were extracted in each sliding window step, and then only 22 relevant ones were selected. The features extracted in each sliding window were fed into the models as well as into “tbbss”/“tssbb” type transfer learning methods. However, this did not help to improve model results and increased running time. In the case of the SARIMAX model, where the extracted relevant traits are taken as exogenous variables, the MAPE was increased by 4-5%. MAPE increased by 4-5% relative to CNN.
12. Multiple regression with regressors, which are features extracted using tsfresh. This approach is inferior to the classical regression AR(1) by 2-3% in accuracy, and also takes 10-14 times longer, because it takes about 10-14 seconds to extract the features in one sliding window (for a window length of 399 values).
13. Catboost, adaboost, based on decision trees. These types of boosting are usually inferior to xgboost by 1-2%, so the following tables show the models from the xgboost package.

In all cases of neural network architectures, 1-3 layer models with 5, 20, 32, 64 and 128 neurons per layer (in different combinations) and activators “linear”, “tanh”, “relu” for each layer were considered. The best results were obtained with the “tanh” activator function on all layers.

3.6. Organizing data for training

The format of the data feed matters a great deal in training. Below are schemes of the description of data feeds in the relevant methods and approaches.

Fig.3 shows a general scheme of training (which is essentially cross validation) and prediction for 1 step ahead on the example of a single window (*window*), which in this case plays the role of the general sample. The interval is divided into subsamples (folds), each subsample includes two parts: *learn* - where training takes place and *target* - the training target, where error calculation and correction of model weights (parameters) are performed. Thus, the model learns to make predictions on a time interval of length T by training on intervals of length L . After the model is trained on *window*, the last T elements are fed into the trained model and an out-sample prediction F is made, where length F is equal to length L . The window is then rolled back and the procedure is repeated again. In Fig. 4 and following $Train = window$, $Test = Out - sample$, in this article $len(window) = 399, len(L) = 5, len(T) = len(F) = len(Test) = 1$.

window					Out-sample	
subsample		...	subsample			
L	T		L	T		
				L	F	

Figure 3: General training scheme.

Fig. 4 shows the data feeding scheme for training models with one gate, boosting and for transfer learning “tbs”, “tsb”, where $i, j \in [btc, snp]$. The

model learns from one row of data, and then builds a prediction either for the same row or for another row – this is how transfer learning takes place.

$train_i$	$test_j$				
	$train_i$	$test_j$			
		...	...		
			...	...	
				$train_i$	$test_j$

Figure 4: Data feed for training single-gate, boosting and TL “tbs”/“tsb” models.

Fig. 5 shows a data feed for training models with two gates. The model is trained in parallel on two rows of data, and then forecasts are made for each of the feeds. This approach is faster than teaching the two networks separately, and also achieves improved Accuracy with a slight decrease in MAPE (less than 1%).

$train_i$	$test_j$				
$train_j$	$test_i$				
	$train_i$	$test_j$			
	$train_j$	$test_i$			
		...	...		
		...	...		
			...	...	
			...	...	
				$train_i$	$test_j$
				$train_j$	$test_i$

Figure 5: Data feed for training models with two gates.

Fig.6 shows the data feed for training models in the case of transfer learning of “tbbss”/“tssbb” type. This approach allows taking into account in one model the patterns of all studied time series, to forecast both series and takes less time than training two different neural networks.

$train_i, train_j$	$test_i, test_j$				
	$train_i, train_j$	$test, test_j$			
		...	...		
			...	...	
				$train_i, train_j$	$test_i, test_j$

Figure 6: Data feed for TL “tbbss”/“tssbb”.

3.7. Benchmark HAR

This article presents the results of the predictions of two HAR models, which performed best in Manevich et al. (2023). The specifications of these two benchmark models are given below. It is important to note that in these models

the variance is predicted and then the square root is taken from it because the task is exactly to forecast the realized volatility.

First, let's define the general notations of the variables used in the models:

1. BPV – realised bipower variation (Barndorff-Nielsen and Shepard (2004)):

$$BPV_t = \sum_{j=1}^{N-1} r_{j,t} r_{j+1,t}.$$

2. RQ – realized quarticity (Corsi et al. (2005)):

$$RQ_t = \frac{N}{3} \sum_{j=1}^N r_{j,t}^4.$$

where $r_{j,t}$ is taken from (1), and the scaling described in (1.1) is also applied.

The benchmarks are the following models:

1. Bitcoin – HARQJ_RV(5, 27)_BPV(4, 21)_RQ(6, 24):

$$\begin{aligned} RV_{t+1} = & \beta_0 + \beta_1 RV_t + \beta_2 \frac{1}{5} \sum_{j=0}^4 RV_{t-j} + \beta_3 \frac{1}{27} \sum_{j=0}^{26} RV_{t-j} \\ & + \beta_4 \max(RV_t - BPV_t, 0) \\ & + \beta_5 \max\left(\frac{1}{4} \sum_{j=0}^3 RV_{t-j} - \frac{1}{4} \sum_{j=0}^4 BPV_{t-j}, 0\right) \\ & + \beta_6 \max\left(\frac{1}{21} \sum_{j=0}^{20} RV_{t-j} - \frac{1}{21} \sum_{j=0}^{20} BPV_{t-j}, 0\right) \\ & + \beta_7 RQ_1 + \beta_8 \frac{1}{6} \sum_{j=0}^5 RQ_{t-j} + \beta_9 \frac{1}{24} \sum_{j=0}^{23} RQ_{t-j} \end{aligned}$$

2. E-mini S&P500 – CHAR_RV(5, 24):

$$RV_{t+1} = \beta_0 + \beta_1 BPV_t + \beta_2 \frac{1}{5} \sum_{j=0}^4 BPV_{t-j} + \beta_3 \frac{1}{24} \sum_{j=0}^{23} BPV_{t-j} + \varepsilon_{t+1}.$$

where RV, BPV, RQ are defined above.

3.8. The Triple Correction Method

This method was designed during the writing of this article and is based on the following ideas:

- The idea of considering a correction for past step deviation (with some weight):

$$Err_j = y_j - \hat{y}_j = \varepsilon_j;$$

- Attempt to consider and reflect the internal dependencies of the original data series and take into account the process changes as a difference of consecutive values (taken with some weight), to correct the algorithm:

$$Izm_j = y_j - y_{j-1};$$

- Attempt to reflect the deviation of the changes in the raw data series from the changes in the series obtained during the algorithm (taken with some weight) to correct the algorithm:

$$ErrIzm_j = ((y_j - y_{j-1}) - (\hat{y}_j - \hat{y}_{j-1})).$$

Let's combine all of them:

$$\begin{aligned} \hat{y}_{t+1} &= \alpha_0 y_t + \alpha_1 (y_t - y_{t-1}) + \alpha_2 (y_t - \hat{y}_t) + \alpha_3 ((y_t - y_{t-1}) - (\hat{y}_t - \hat{y}_{t-1})) = \\ &= \alpha_0 y_t + \alpha_1 Izm_t + \alpha_2 Err_t + \alpha_3 ErrIzm_t = \\ &= \alpha_0 x_{0,t} + \alpha_1 x_{1,t} + \alpha_2 x_{2,t} + \alpha_3 x_{3,t}, \end{aligned}$$

where $\hat{y}_{t+1}$ – the predicted value at time $t+1$, y_j – the real “historical” data, $\hat{y}_j$ is the predicted data ($j = 0, \dots, t$), $\hat{y}_{0,1} = y_{0,1}$, α_k – the adaptation parameters, $k = 0, 1, 2, 3$.

The vector of optimal α_i is found by the following algorithm:

$$\begin{aligned} \sum_{j=0}^{t-1} (y_{j+1} - \hat{y}_{j+1})^2 &= \sum_{j=0}^{t-1} (y_{j+1} - \alpha_0 x_{0,j} + \alpha_1 x_{1,j} + \alpha_2 x_{2,j} + \alpha_3 x_{3,j})^2 \rightarrow \min; \\ \sum_{j=0}^{t-1} x_{i,j} (y_{j+1} - \alpha_0 x_{0,j} + \alpha_1 x_{1,j} + \alpha_2 x_{2,j} + \alpha_3 x_{3,j}) &= 0, \text{ where } i = 0, \dots, 3; \end{aligned}$$

Matrix form:

$$Y_{t-1}^T Y_{t-1} \alpha = Y_{t-1}^T X_{0,t}$$

$$\alpha = (Y_{t-1}^T Y_{t-1})^{-1} Y_{t-1}^T X_{0,t},$$

where $Y_{t-1} = [X_{0,t-1}, X_{1,t-1}, X_{2,t-1}, X_{3,t-1}]$, and Y_{t-1}^T is the transpose of matrix Y_{t-1} . When solving the system in the first 4 iterations (due to the sparse matrix Y), regularization is applied with the addition of 1 on the main diagonal.

$$X_{i,t-1} = \begin{bmatrix} x_{i,0} \\ x_{i,1} \\ \vdots \\ x_{i,t-1} \end{bmatrix}, X_{0,t} = \begin{bmatrix} x_{0,1} \\ x_{0,2} \\ \vdots \\ x_{0,t} \end{bmatrix}, \alpha = \begin{bmatrix} \alpha_0 \\ \alpha_1 \\ \alpha_2 \\ \alpha_3 \end{bmatrix}$$

Thus, iteratively, up to the last known value at step t , all 4 coefficients α_i are obtained, which are then substituted into the original equation to obtain the prediction $\hat{y}_{t+1}$.

Thus, an iterative adaptive method with four adaptation parameters is obtained – the “Triple Correction Method” or abbreviated “TCM”. This is the form in which it is further used in the implemented algorithms.

We also propose a modification of this method the “Corrected Triple Correction Method” (CTCM). The modification is that at the last iteration the obtained predicted $t + 1$ value is multiplied by $(1 + \text{persentile}(MPE_t, 80))$, where $\text{persentile}(MPE_t, 80)$ is the 80th persentile of all $MPE_j = \frac{RV_j - h_j}{RV_j}$, $j = 0, \dots, t$. The 80th persentile is taken from empirical considerations and is in general a parameter of the CTCM model.

As a result, for CTCM the forecast at the moment $t + 1$ is calculated by the formula:

$$\hat{y}_{t+1} = (\alpha_0 x_{0,t} + \alpha_1 x_{1,t} + \alpha_2 x_{2,t} + \alpha_3 x_{3,t}) (1 + \text{persentile}(MPE_t, 80))$$

Thus, the correction allows to partially account for the percentage error made in the previous fitting steps and improve the prediction result. It is important to note that uncertainties for the adjustment are counted with the sign. Both positive and negative MPE values should be taken into account.

3.9. Methodology for model comparison

In Manevich et al. (2023), the MCS test is used to compare a large number of models. In this article, a much smaller number of models are compared and the quality of the predictive power of the model is evaluated using the average prediction error of 1 step forward. As practice has shown, the MCS test and the comparison by the value of the chosen metric show almost identical results when it comes to 1-step-ahead forecasting.

The error metric used is:

$$MAPE = \frac{100\%}{K} \sum_{j=1}^K \frac{|RV_j - h_j|}{RV_j}, \quad (2)$$

where h_j – forecast at time j , RV_j – value of realized volatility on day j . In this article, all forecasts are made 1 step ahead 170 times, i.e. $K = 170$.

Also of interest is the Accuracy indicator – how well the model predicts a rise or fall in the values of the studied series. A fall is denoted by 0, a rise by 1. Thus:

$$Accuracy = \frac{1}{K} \sum_{j=1}^K change_j \quad (3)$$

$$change_j = \begin{cases} 1, & \text{if } RV_j > RV_{j-1} \\ 0, & \text{if } RV_j < RV_{j-1} \end{cases}$$

where $change_j$ – the indicator function showing whether the values of the series increased or decreased at time j .

4. Results

◆ The following tables present in the corresponding columns the model name, the MAPE percentage for the model, and the Accuracy score. The rows with ARIMA models present the classical variation of the ARIMA model (1, 0, 0), as well as the models with the best MAPE and the best Accuracy. No transfer learning is applied to the ARIMA models. Empirically, it is found that in this article the MA part does not affect (or practically does not affect) the results of ARIMA models, so in the tables the model is specified with $q = 0$.

Also, the results for both processes showed that reducing the length of the windows has a negative effect on the prediction results. Thus, the sliding window length of 399 values in this article is the best.

4.1. Bitcoin

In Manevich et al. (2023), the best error in the MAPE metric is HARQJ_RV (5, 27)_BPV (4, 21)_RQ (6, 24), and it and its error are taken as the benchmark in this study. Table 2 shows the MAPE errors of the best error models from all the studied model families and the benchmark. The CNN models with one layer, one gate, and one feature perform best and give an error of 25.28% (the results may vary slightly when the code is replayed). The results of the CNN models differ little from each other, while they significantly outperform the results of the other architectures described earlier. Boosting and ARIMA are inferior to CNN in precision, but in some cases show slightly higher Accuracy scores. Also, worth noting is CTCM showed almost the same results as CNN, but the algorithm took only about 10 seconds, while the entire process of training, prediction and re-runs of the neural network took about 10 minutes. Single-layer CNNs, among neural networks, are the most suitable for forecasting such a highly volatile asset as Bitcoin. Also, their big plus is the speed, they are much faster than other considered neural network models. In general, CTCM model is the best choice both in terms of MAPE and speed, though it can be slightly inferior in Accuracy.

Model	MAPE, %	Accuracy
HARQJ_RV(5, 27)_BPV(4, 21)_RQ(6, 24)	29.51	0.51
ARMA (1, 0)	27.69	0.67
ARMA (5, 0)	27.20	0.63
ARMA (2, 0)	27.24	0.67
M_1_CNN_TS_1	25.28	0.62
M_2_CNN_TS_1	25.40	0.65
M_1_CNN_TS_2	25.84	0.64
M_2_CNN_TS_2	26.21	0.63
xgboost_0.2_0.3_8	27.01	0.59
xgboost_0.9_0.1_2	31.22	0.67
TCM	28.20	0.64
CTCM	25.33	0.63

Table 2: Model errors for RV Bitcoin.

Fig. 7 shows graphs of predictions on pending intervals of realized volatility of Bitcoin, benchmark HAR model, single-layer CNN model, CTCM and the boosting that showed the best results among all the boostings. Models with one time series on the gate were better in MAPE, and the presented boosting showed the best results in Accuracy, that's why they are presented on the chart. CTCM, CNN and HAR models behave similarly in intervals without strong peaks, but in the case of strong jumps CNN and CTCM show better adaptive capabilities. Boosting behaves similarly to CNN but significantly overestimates, such as the December 2021 peak.

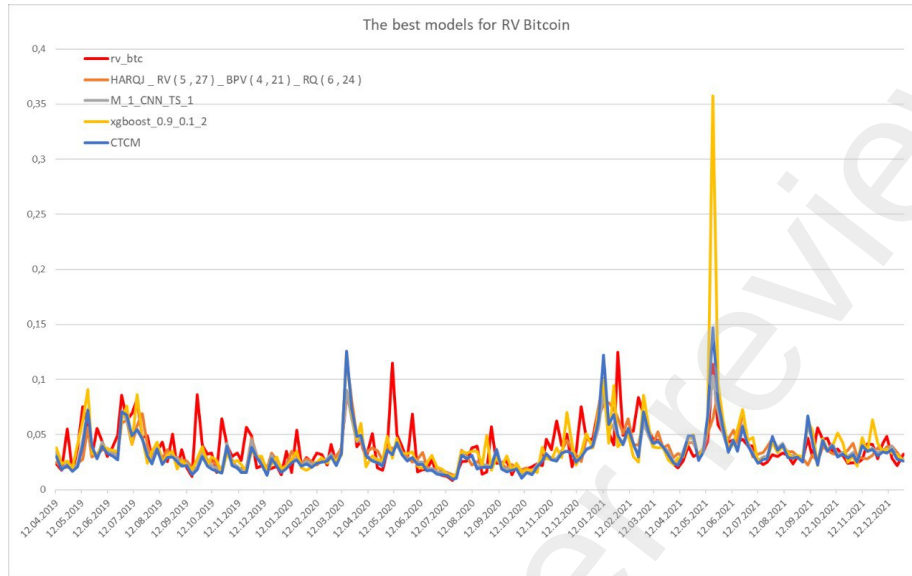


Figure 7: The red line – Bitcoin RV, the others – predictions of the corresponding models.

4.2. E-mini S&P 500

In Manevich et al. (2023), the best error in the MAPE metric is the CHAR_RV(5, 24) benchmark model. Table 3 presents the MAPE errors of the best error models from all of the model families studied and the benchmark for the RV E-mini S&P 500. The best model by MAPE and by Accuracy was ARMA (9, 0). In this case, models with two gates perform slightly better than models with one gate. This is due to the fact that the network learns two assets at once, one of which has significantly higher volatility, which allows us to take more behavioral patterns into account when forecasting. At the same time, these models for Bitcoin show good Accuracy values and are slightly inferior in terms of MAPE, which indicates their applicability in volatility prediction tasks. The model with two attributes is comparable to the model with one attribute, but it allows getting forecasts for two time series at once. All presented models outperform models with other architectures but have only a slight advantage over the HAR model, while the classical ARMA model shows errors of

almost 5%. It is also important to mention CTCM, which also outperforms all algorithms except ARMA (9, 0) in MAPE but is slightly inferior in Accuracy.

Model	MAPE, %	Accuracy
CHAR_RV(5, 24)	36.12	0.54
ARMA (1, 0)	34.95	0.60
ARMA (9, 0)	30.74	0.66
M_1_CNN_TS_1	35.43	0.50
M_2_CNN_TS_1	35.42	0.52
M_1_CNN_TS_2	35.52	0.60
M_2_CNN_TS_2	35.36	0.60
xgboost_0.9_0.8_2	34.69	0.61
xgboost_0.4_0.6_6	36.40	0.66
TCM	33.70	0.57
CTCM	32.05	0.57

Table 3: Model errors for RV E-mini S&P 500.

Fig.8 shows forecasts on the deferred interval of bitcoin volatility, the benchmark HAR model, CTCM, xgboost_0.9_0.8_2 and ARMA (9, 0). All models are poor at capturing consecutive peaks and mostly reflect only the second peak, almost ignoring the first one.

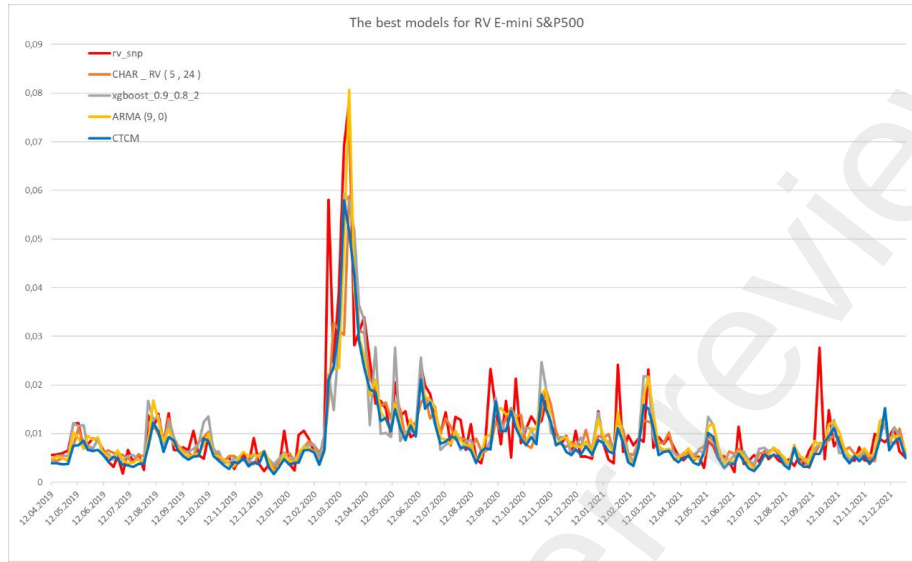


Figure 8: The red line – RV E-mini S&P 500, the others – the forecasts of the corresponding models.

4.3. Transfer Learning

Tables 4 and 5 show the MAPE and Accuracy of the corresponding models using TL. Very interesting results are observed:

1. For the “tsb” transfer there is a 2% decrease in prediction accuracy and an increase in Accuracy. MAPE is the main metric of model quality, so this variant of learning is worse than learning without transfer. This can be explained by the fact that when training is transferred from a less volatile asset to more volatile asset, the models do not respond well to the transfer. This leads to a decrease in prediction precision;
2. For the neural network, the tbc transfer allowed to improve MAPE by 2.5% with a slight decrease in Accuracy. This can be explained by the fact that the neural network was trained on a much more volatile asset, which shows a larger range of behavior patterns, which allowed us to improve the results;

3. For boosting the “tbs” transfer showed a 2-3x increase in MAPE, although Accuracy is comparable to the other models;
4. The “tbbss” transfer in neural networks for Bitcoin showed a slight decrease in MAPE and an increase in Accuracy. Adding another, less volatile asset slightly worsens the result in the case of this transfer learning for neural networks;
5. The “tbbss” transfer in neural networks for E-mini S&P 500 shows more interesting results. There is a decrease in MAPE relative to models without transfer learning, but this figure is worse than that obtained in the case of “tbs” transfer;
6. Boosting with “tbbss” transfer for Bitcoin improved MAPE and Accuracy by 1-2%. However, for E-mini S&P 500 there is a 1-2% increase in MAPE and Accuracy.

Model	MAPE, %	Accuracy
M_1_CNN_TS_1_tsb	27.76	0.66
M_2_CNN_TS_1_tsb	27.94	0.66
M_1_CNN_TS_1_tbbss	25.83	0.67
M_2_CNN_TS_1_tbbss	25.90	0.65
xgboost_0.4_0.4_2_tsb	34.78	0.59
xgboost_0.6_0.6_1_tsb	39.35	0.68
xgboost_0.1_0.9_3_tbbss	25.60	0.64
xgboost_0.9_0.4_1_tbbss	29.81	0.70

Table 4: Model errors for RV Bitcoin.

Model	MAPE, %	Accuracy
M.1_CNN_TS.1_tbs	32.82	0.59
M.2_CNN_TS.1_tbs	33.14	0.58
M.1_CNN_TS.1_tbbss	33.75	0.59
M.2_CNN_TS.1_tbbss	34.08	0.57
xgboost_0.9_0.4_1_tbs	86.46	0.57
xgboost_0.6_0.4_7_tbs	129.88	0.64
xgboost_0.1_0.2_3_tbbss	35.49	0.60
xgboost_0.5_0.6_10_tbbss	38.84	0.67

Table 5: Model errors for RV E-mini S&P 500.

This approach produces improved Accuracy and is comparable to the simple MAPE approach, but requires caution in the application, as incorrect transposition can lead to increased errors.

4.4. Prediction errors

This section presents error plots for the best performing models as well as for the CTCM model since their results are very similar to the best models and only for snp 1.5% inferior to ARMA (9, 0).

For btc, in Fig. 9, boosting is characterized by sharp spikes in error. For example, in August-September 2020, the MAPE was about 200%. At the same time, the errors given for the neural network, as well as for CTCM, show relative stability and even stationarity, which allows us to speak about greater stability and precision of these models, and, consequently, greater applicability.

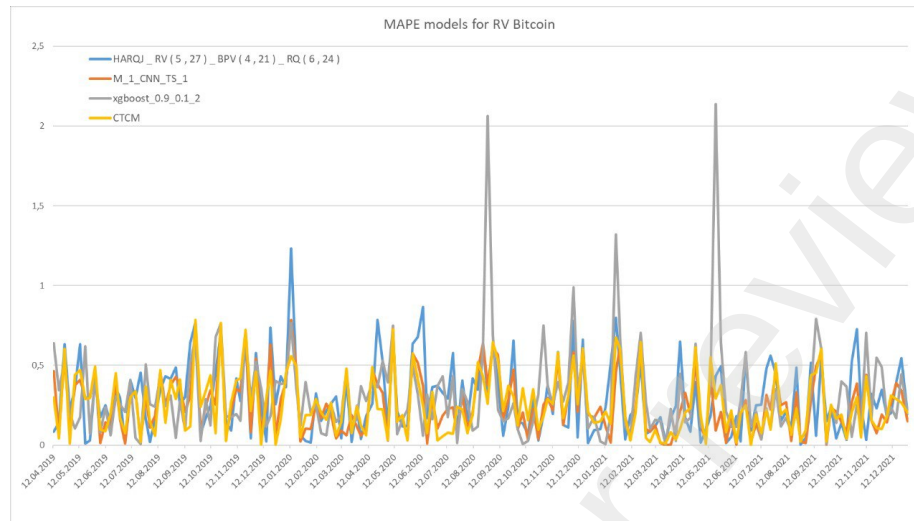


Figure 9: MAPE models with the smallest errors for Bitcoin RV.

Fig. 10 shows significantly more frequent error spikes for snp than for btc. As can be seen from the graph, the CHAR family model tends to overestimate the predicted value more often. CTCM shows higher precision and has the lowest MAPE value given on almost all spikes.

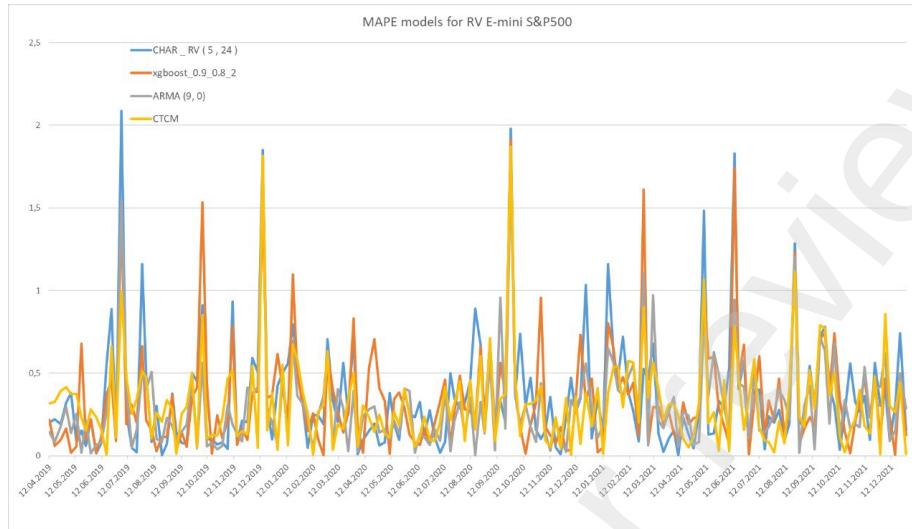


Figure 10: MAPE models with the smallest errors for the RV E-mini S&P 500.

Although errors are large in both cases, CTCM shows more robust MAPE behavior than neural network models, CHAR, and boostings, although it is slightly inferior to classical ARMA (9, 0) in one case.

5. Conclusion

In conclusion:

1. The results for RV Bitcoin show that the use of neural network models can reduce the prediction error by 4% relative to the benchmark, which is a good practical result.
2. Results for RV E-mini S&P 500 show that the use of neural network models slightly reduces the error - by 1-2%, while the ARMA family model reduced the error by 5%.
3. For both time series under study, the CTCM model shows good MAPE results, comparable to the results of the best selected neural network models. This indicates the potential of the method and the promise of its use in further research.

4. The boostings show improved Accuracy scores and comparable MAPE results to neural network models. This shows their applicability in volatility forecasting.
5. The best results among neural networks are shown by CNN models. Moreover, single-layer models with one gate and one feature show the highest precision. This also confirms the results obtained by other researchers, who obtained that CNN models are the best for time series forecasting.
6. For the RV E-mini S&P 500 the CNN architecture with two features shows better results than the model with one gate feature. It can be explained by the fact that the network learns two assets at once, one of which has significantly higher volatility and allows to take more behavioral patterns into account when making forecasts. This approach saves twice as much time to get a forecast for both data series, allowing to get almost the same forecasts for the more volatile asset and an improvement in precision for the less volatile one.
7. LSTM and MLP models show a lag with respect to real data, which was also previously observed in the articles of other researchers. This suggests poor applicability of these architectures to financial time series forecasting tasks. At least, in the case of small training sample sizes. Graphs and errors of these models are not given in the article, since their results are significantly inferior to all the given models.
8. For neural networks the transfer learning from a more volatile asset to a less volatile one allowed to increase the prediction accuracy of the less volatile asset. At the same time, the reverse transfer of training for both neural networks and boosting (from less volatile to more volatile) showed a decrease in forecasting accuracy.
9. For boosting, the transfer learning from a more volatile asset to a less volatile one (“tbs”) gave a significant decrease in prediction precision.
10. Learning on both assets simultaneously (not to be confused with feeding two time series to the gate) and further transferring to both time series gives results close to the results of single-layer CNNs for both time series.

This indicates the potential of this learning method and requires a more detailed study of a larger number of assets.

11. For boosting, training on both assets simultaneously and further rollover for Bitcoin increased MAPE and Accuracy by 1-2%. However, for E-mini S&P 500 there is a 1-2% increase in MAPE and Accuracy. This result is ambiguous and needs to be studied in more detail.
12. All of the results obtained in the Transfer Learning section are relevant to these processes. The detected effect requires additional research on a larger sample of assets for complete representativeness of the results and confirmation of the hypothesis proposed. This will be done in further research.
13. The analysis of forecasting errors reveals that CTCM models have a smaller spread of errors than neural network models and boostings. Also for this model, there are no sharp spikes in MAPE for Bitcoin and smaller error spikes for E-mini S&P 500 than for other models.
14. Although neural network architectures and machine learning models produced more precise results than classical models. Neither of the models produced nearly applicable results – the lowest MAPEs were 25.28% and 30.74% for Bitcoin and E-mini S&P 500, respectively. The best Accuracy for both processes is 0.67.

The results obtained in this article show that even in spite of the small length of intervals for training and aggregation specificity of the realized volatilities, the neural network models showed results superior to the classical HAR models without a detailed selection of hyperparameters, although in the case of snp they were inferior to ARMA and CTCM.

So it is not always possible to speak about the applicability and unambiguous superiority of neural network models over classical ones. Neural network models and machine learning methods are more demanding in the selection of hyperparameters, architecture and also have a longer running time. All this leads to the idea that one way to improve forecasting results is to combine

different methods, which will be the focus of our future research.

Acknowledgements

The article was prepared within the framework of the HSE University Basic Research Program.

References

- Barndorff-Nielsen O. E., Shepard N. (2004). Power and bipower variation with stochastic volatility and jumps. *Journal of Financial Econometrics*, 2 (1), 1–37. DOI: 10.1093/jjfinc/nbh001.
- Bollerslev T. (1986). Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31, 307–327. DOI: 10.1016/0304-4076(86)90063-1.
- Christ M., Braun N., Neuffer J., Kempa-Liehr A.W. (2018). Time Series Feature Extraction on basis of Scalable Hypothesis tests (ts-fresh – A Python package). *Neurocomputing* 307 (2018) 72-77, DOI: 10.1016/j.neucom.2018.03.067.
- Corsi F., Mittnik S., Pigorsch C., Pigorsch U. (2005). The volatility of realized volatility. CFS Working article, No. 2005/33, Goethe University Frankfurt. DOI: 10.1080/07474930701853616.
- Dutta A., Bouri E., Saeed T. (2021). News-based equity market uncertainty and crude oil volatility. *Energy*, 222, Article 119930. DOI: 10.1016/j.energy.2021.119930.
- Fischer T., & Krauss C. (2018). Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, 270(2), 654–669. DOI: 10.1016/j.ejor.2017.11.054.
- Hochreiter S., & Schmidhuber J. (1997). Long Short-Term Memory. *Neural Computation*; 9 (8): 1735–1780. DOI:10.1162/neco.1997.9.8.1735

- Kim H. Y., & Won C. H. (2018). Forecasting the volatility of stock price index: A hybrid model integrating LSTM with multiple GARCH-type models. *Expert Systems with Applications*, 103, 25–37. DOI: 10.1016/j.eswa.2018.03.002.
- Kristjanpoller W., & Minutolo, M. C. (2016). Forecasting volatility of oil price using an artificial neural network-garch model. *Expert Systems with Applications*, 65, 233–241. DOI: 10.1016/j.eswa.2016.08.045
- Kristjanpoller, W., & Minutolo, M. C. (2018). A hybrid volatility forecasting framework integrating garch, artificial neural network, technical analysis and principal components analysis. *Expert Systems with Applications*, 109, 1–11. DOI: 10.1016/j.eswa.2018.05.011
- Livieris I. E., Pintelas E., Pintelas P. (2020). A CNN-LSTM model for gold price time-series forecasting. *Neural Computing and Applications*. DOI: 10.1007/s00521-020-04867-x.
- Livieris I. E., Kiriakidou N., Stavroyiannis S., Pintelas P. (2021). An Advanced CNN-LSTM Model for Cryptocurrency Forecasting. *Electronics*, 10(3), 287. DOI: 10.3390/electronics10030287.
- Manevich V. A., Peresetsky A. A., Pogorelova P. V. (2022). Stock market and cryptocurrency market volatility. *Applied Econometrics*. 65, 65–76. DOI: 10.22394/1993-7601-2022-65-65-76. (in Russian).
- Manevich V. A., Aganin A., Peresetsky A. A., Pogorelova P. V. (2023). Comparison of cryptocurrency and stock market volatility forecast models. *HSE Economic Journal*. (to appear)
- Shen Z., Wan Q. and Leatham D. J. (2021). Bitcoin return volatility forecasting: A comparative study between GARCH and RNN. *Journal of Risk and Financial Management*, 14(7), 337. DOI:10.3390/jrfm14070337.
- Shusheng D., Tianxiang C., Yongmin Z. (2022). Futures volatility forecasting based on big data analytics with incorporating an order imbalance effect. In-

ternational Review of Financial Analysis Volume 83, October 2022, 102255.
DOI: 10.1016/j.irfa.2022.102255.

Singhal, D., & Swarup, K. (2011). Electricity price forecasting using artificial neural networks. *International Journal of Electrical Power & Energy Systems*, 33(3), 550–555. DOI: 10.1016/j.ijepes.2010.12.009.

Zolfaghari, M., & Gholami, S. (2021). A hybrid approach of adaptive wavelet transform, long short-term memory and ARIMA-GARCH family models for the stock index prediction. *Expert Systems with Applications*, 182, 115149. DOI: 10.1016/j.eswa.2021.115149.

Wang Y. and Guo Y. (2020). Forecasting method of stock market volatility in time series data based on mixed model of arima and xgboost. *China Communications*, 17(3), 205–221. DOI: 10.23919/JCC.2020.03.017

Wang, L., Ma, F., Hao, J., & Gao, X. (2021). Forecasting crude oil volatility with geopolitical risk: Do time-varying switching probabilities play a role? *International Review of Financial Analysis*, 76, Article 101756. DOI: 10.1016/j.irfa.2021.101756

Software packages

Adaboost
Catboost
Finam
Keras
My GitHub
Optuna
Scikit-learn
TabNet
Tensorflow
Tsfresh
Xgboost